Informe Analítico de Rendimiento y Estrategias de Optimización - TP5

Autor: Máximo Quiroga
Institución: Universidad Tecnológica Nacional (UTN) - Facultad Regional Mendoza

Introducción

El presente informe documenta las métricas de rendimiento y las estrategias de optimización aplicadas sobre la base de datos transaccional. Se analizan los costos de entrada/salida (I/O), el impacto de las estructuras de acceso (índices B-Tree), y el uso de vistas tanto lógicas como materializadas para la separación de cargas OLTP y OLAP.

Parte A: Plan de Indexado

1. Costo en lecturas (Consultas)

La implementación de índices B-Tree reduce la complejidad de búsqueda de secuencial (O(N)) a logarítmica (O(log N)), minimizando drásticamente la cantidad de bloques leídos del disco.

Escenario de Búsqueda	Plan y Tiempo ANTES	Índices Aplicados	Plan y Tiempo DESPUÉS	Mejora Factor
Búsqueda por nombre	`Seq Scan` (12.537 ms)	`idx_productos_nombre`	`Index Scan` (**0.162 ms**)	~77x rápido
Detalles por precio	`Parallel Seq Scan` (97.198 ms)	`idx_detalles_precio`	`Bitmap Heap Scan` (**43.534 ms**)	~2.2x rápido

Nota del Profesor: Es destacable el uso del `Bitmap Heap Scan`. Cuando el motor identifica que un índice retornará múltiples filas (baja selectividad relativa pero aún útil), construye un mapa de bits en memoria con las páginas a visitar. Esto evita leer la misma página de disco múltiples veces, optimizando el I/O aleatorio.

2. Costo en escrituras (INSERTs)

Operación: Inserción masiva (5000 registros en `detalles_pedido`)

Métrica	Estado Anterior	Estado Post-Indexado	Delta (Impacto)
Tiempo Total	1218.459 ms	1579.304 ms	+360.845 ms (+29%)

Conclusión empírica y técnica: Los índices aceleran las lecturas drásticamente, pero penalizan el rendimiento de escritura por la obligatoriedad matemática de actualizar el árbol B interno del motor. Además, por cada `INSERT`, el motor de la base de datos no solo escribe en la tabla (Heap), sino que puede provocar "page splits" (divisiones de página) y mayor carga en el Write-Ahead Log (WAL).

3. Justificación de índice descartado por Sobreindexación

Índice propuesto y rechazado: `CREATE INDEX idx_pedidos_forma_pago ON pedidos(forma_pago);`

Justificación técnica: Baja cardinalidad. La columna solo posee 3 estados. Cada búsqueda retorna ~33% de los datos. El motor preferirá un Seq Scan al ser tan masivo el retorno de bloques. Crearlo es sobreindexar, ralentizando escrituras sin obtener ningún provecho en lecturas.

Sugerencia adicional: Si existiera un estado altamente consultado pero poco frecuente (ej. 'RECHAZADO' siendo el 1% de los casos), la solución óptima sería un Índice Parcial (WHERE forma_pago = 'RECHAZADO').

Parte B: Vistas Simples y Equivalencia

Las vistas simples operan como macros lógicas. Es fundamental recordar que no mejoran el rendimiento por sí mismas, ya que el motor las reescribe ("query rewriting") y ejecuta la consulta subyacente. Su valor radica en la seguridad, la abstracción y la simplificación del código de aplicación.

Verificación de la vista v_productos_vigentes:
Para dar por válida la vista según indica la cátedra, simulamos una validación cruzada mediante EXCEPT comparando el resultado de consultar la vista (SELECT * FROM v_productos_vigentes) contra la consulta relacional subyacente. El resultado retorna exactamente 0 filas anómalas, demostrando 100% de equivalencia matemática y lógica.

Parte C: Vistas Materializadas

Las vistas materializadas almacenan físicamente el resultado de la consulta en el disco, actuando como una caché precalculada. Esto es vital para analítica en sistemas que no deben penalizar la carga transaccional principal.

Reporte: Facturación por categoría y mes

Método de Consulta	Operación Principal	Tiempo Ejecución
Consulta Cruda (Sin materializar)	`HashAggregate` (4 tablas)	1065.695 ms
Vista Materializada	`Seq Scan` (Caché en disco)	**0.037 ms**

Mejora de Rendimiento: ~28.800x más rápido.

Estrategia de Refresco (REFRESH)

Dado que este reporte totaliza facturación agrupada a nivel gerencial, los meses anteriores son inmutables por naturaleza; únicamente el mes en curso sufre modificaciones diarias.

Frecuencia de refresco recomendada: Ejecutar un REFRESH MATERIALIZED VIEW CONCURRENTLY en horario valle (ej. 03:00 AM) de forma automática mediante un Cron Job en la base de datos.

Nota técnica crítica: El modificador CONCURRENTLY es indispensable aquí. Permite que la vista se actualice sin aplicar un bloqueo exclusivo (exclusive lock), lo que significa que los usuarios pueden seguir leyendo datos mientras la vista se recalcula en segundo plano.

Impacto en los usuarios: Para la gerencia, significa que los datos de "facturación acumulada del mes en curso" tendrán una latencia o desfase (staleness) máximo de 24 horas. Este comportamiento es un estándar perfectamente aceptable en la industria de reportes, y salva a la base de datos transaccional (OLTP) de congelarse durante ~1 segundo cada vez que alguien solicita el reporte.